FreezeFrame

Catching scam money across banks, in real time, without any bank seeing another bank's customers.

Shared confidential enclave on IBM LinuxONE, IBM Snap ML Graph Feature Preprocessor, Snap ML scoring on the IBM Z AI accelerator, IBM Secure Execution design

THE PROBLEM

Mule chains cross banks. Each bank sees one slice.

- A scam victim's payment hops through **money mules at several banks within minutes**, then it's gone.
- Each bank sees only its own payments, so **no one sees the whole chain**.
- Banks **can't pool raw customer data**: privacy law, bank secrecy, competition, new exposure.
- Many mules are unwitting, recruited through romance, job and lottery scams.

$20B+

cybercrime losses reported to the FBI in 2025 (FBI IC3 report)

58%

of the money in play was frozen when banks acted fast via the FBI Recovery Asset Team (2025). Speed decides recovery.

Sources: FBI IC3 2025 report; ZIGRAM analysis of IC3 2025 (see design doc)

THE SOLUTION

Pool the payments, not the data

Bank A
own payments only

Bank B
own payments only

Bank C
own payments only

verifies enclave attestation first (code + model digests)

→ signed + sealed

ENCLAVE, IBM LINUXONE, SECURE EXECUTION

1, Verify
signature, owner, freshness, replay

2, Pseudonymise
HMAC on account numbers before any processing

3, Graph patterns, live
Snap ML Graph Feature Preprocessor: cycles, fan-in/out, scatter-gather

4, Score + decide
Snap ML booster (Z AI accelerator) → release, hold, freeze; hold at next hop

Warm tier re-scores with all patterns every few seconds and escalates

→ signed + sealed

To the paying bank only
decision + reason codes ("money moving in a cycle")

To the receiving bank
freeze request naming only its own account

To everyone
consortium counts, latency, security events

Federated learning would not close this gap: each bank's graph features would still come from its own slice

RESULTS, IBM AMLWORLD HI-SMALL, 5M PAYMENTS, 1,015,669-PAYMENT TEST SPLIT

Together catches more, with almost no false freezes

MINORITY-CLASS F1

0.559 → 0.630

each bank alone (mean of 3 random deals of banks to 5 members, 0.535–0.580) → together in the enclave. Published GFP + XGBoost baseline: 0.632.

LAUNDERING PAYMENTS FROZEN AUTOMATICALLY

344–389 → 611

of 1,797 laundering payments. Together, only **12 legitimate payments** were frozen in the whole test split.

Member (seed 0)	F1 alone	F1 together
A	0.361	0.420
B	0.593	0.666
C	0.605	0.665
D	0.621	0.647
E	0.609	0.712

STREAMING LATENCY (SCHEDULED ARRIVAL → PUBLISHED DECISION)

p99 under 50 ms up to **539 payments/s** with the warm tier running, **2,448/s** hot path alone. CPU scoring; the IBM Z AI accelerator is used automatically when present.

All numbers from results/HI-Small in the repository

Every attack we could think of, blocked and tested

Attack	Defence
Read another bank's payments or decisions	Sealed to the recipient's key; decisions go only to the owner
Submit payments as another bank	Ed25519 signature against the member's registered key
Authenticated bank submits someone else's payment	Ownership check: sending bank must belong to the member
Replay a captured message	Nonce + 120 s freshness window
Forge a "release" decision on a bank's topic	Banks accept only enclave-signed messages
Overwrite a bank's key / drain its freeze notices	One-time enrollment token; signed requests; signed key rotation
Talk to a fake or modified enclave	Attestation: keys + code measurement + model digests, checked by every bank
Learn account numbers from features, logs, models	HMAC pseudonyms before processing; tests scan for raw strings

Reason codes explain holds by **pattern family** only. The live dashboard lets judges run 5 attacks and watch each one get blocked. 31 automated tests.

Full threat model in SECURITY.md

WHY IBM Z

The platform is the design

Need	IBM Z / LinuxONE
Mine laundering patterns as payments stream	IBM Research's **Graph Feature Preprocessor** in Snap ML (AI Toolkit for IBM Z and LinuxONE)
Score every payment in milliseconds	Snap ML tree inference on the **IBM Z Integrated Accelerator for AI** (zDNN, z16/z17), CPU fallback
An enclave no single party controls	**IBM Secure Execution** + Confidential Computing Container Runtime (Hyper Protect) with attestation
Score where payments already run	IBM Z handles most of the world's payment transactions; Snap ML ships **s390x** wheels
Bank-grade streaming	Kafka transport compatible with **IBM Event Streams**
Realistic data without real customers	**IBM AMLworld** synthetic transactions (5M → 180M sizes for scale tests)

Dockerfile + LinuxONE Community Cloud runbook included (deploy/)

IMPACT AND NEXT STEPS

Who benefits

- **Scam victims:** money stopped in the first hops, while recovery is still possible.
- **Unwitting mules:** flagged early, often victims themselves.
- **Banks:** detection power of a shared view, without sharing data or breaching privacy law.
- **Regulators:** explainable, auditable decisions, with every number traceable to results files.

NEXT

- Accelerator vs CPU latency on z16/z17
- Bind keys to a hardware Secure Execution attestation
- Scale test on HI-Medium (32M payments)
- Per-member Kafka ACLs; IBM Event Streams

TRY IT IN MINUTES

```
pip install -e '.[dev,xgb]'
./scripts/quick_demo.sh
```